Recursive Problems
Trace the Function Calls

Problem 01

What will be displayed on the screen if the following recursive function **F(5)** is called from the main function? Also, clearly show the trace of each function call with the value of **n** passed to it, as well as the return value at each step.

Function Code	Trace of Function calls

`int F(int n)`
`{`
`  if (n <= 0)`
`  return 0;`
`  cout << n << " ";`
`  return F(n - 1);`
`}`
 	
`int main()`
`{`
`  F(5);`
`  return 0;`
`}`	

Output:	

Problem 02

What will be displayed on the screen if the following recursive function **F(3, 2)** is called from the main function? Also, clearly show the trace of each function call with the values of **n** and **a** passed to it, as well as the return value at each step.

Function Code	Trace of Function calls

`int F(int n, int a)`
`{`
`  if (n <= 0)`
`  return a;`
`  cout << n + a << " ";`
`  return F(n - 1, a + 1);`
`}`
 	
`int main()`
`{`
`  F(3, 2);`
`  return 0;`
`}`	

Output:	

Problem 03

What will be displayed on the screen if the following recursive function **F(4, 0, 1)** is called from the main function? Also, clearly show the trace of each function call with the values of **n**, **a** and **b** passed to it, as well as the return value at each step.

Function Code	Trace of Function calls

`int F(int n, int a, int b)`
`{`
`  if (n <= 0)`
`  return a + b;`
`  cout << a << "," << b << " ";`
`  return F(n - 1, b, a + b);`
`}`
 	
`int main()`
`{`
`  F(4, 0, 1);`
`  return 0;`
`}`	

Output:	

Question # 04

What will be displayed on the screen if the following recursive function **F(3, 1, 2)** is called from the main function? Also, clearly show the trace of each function call with the values of **n**, **a** and **b** passed to it, as well as the return value at each step.

Function Code	Trace of Function calls

`int F(int n, int a, int b)`
`{`
`  if (n <= 0)`
`  return a - b;`
 	
`  cout << n << " " << a << " " << b << " ";`
 	
`  int x = F(n - 1, a + 1, b + 2);`
`  int y = F(n - 2, a + 2, b + 3);`
 	
`  return x + y;`
`}`
 	
`int main()`
`{`
`  F(3, 1, 2);`
`  return 0;`
`}`	

Output:	

Recursive Problems
Write the Code

Problem 1: Calculate the Power of a Number

Write a recursive function that takes two integers, **base** and **exponent**, and returns the result of raising **base** to the power of **exponent**.

Parameters:

- **base:** The number to be raised to the power of the **exponent**.
- **exponent:** The exponent to which the **base** is raised.

Example:

Input: base = 3, exponent = 4 **Output:** 81

Problem 2: Check if an Array is Sorted

Write a recursive function that checks whether a given array of integers is **sorted in ascending order**.

The function should take the **array** and its **size** as parameters.
It should return **true** if the array is sorted in ascending order, and **false** otherwise.

Example:

Input: {2, 4, 6, 8, 10} **Output:** true
Input: {5, 3, 7, 9} **Output:** false

Problem 3: Find Maximum Element in an Array

Write a recursive function to **find** and **return** the **maximum element** in an array of integers.

The function should take the **array** and its **size** as parameters.
It should return the **largest element** present in the array.

Example:

Input: {12, 45, 23, 67, 34} **Output:** 67

Problem 5: Reverse an Array

Write a recursive function that **reverses** an array of integers.

The function should take the **array** and its **size** as parameters.
It should modify the array so that its elements are in **reverse order**.

Example:

Input: {1, 2, 3, 4, 5} **Output:** {5, 4, 3, 2, 1}

Problem 6: Generate All Subsets of a Set

Write a recursive function to **generate all subsets** of a given set of integers.

The function should take an **array** of integers and its **size** as parameters.
It should **return a list or an array** containing **all possible subsets** of the input set.

Example:

Input: {1, 2, 3}

Output: { {}, {1}, {2}, {3}, {1, 2}, {1, 3}, {2, 3}, {1, 2, 3} }